Lecture 7. Strings

stringr basics, regular expressions, tools for matching

Contents

7.1 String basics **2**
- 7.1.1 Length, combining, subsetting . . . 2

7.2 Regular expressions **3**
- 7.2.1 Basic matches . . . 3
- 7.2.2 Anchors and character classes . . . 3
- 7.2.3 Repetition . . . 4
- 7.2.4 Grouping and backreferences . . . 4

7.3 Tools **4**
- 7.3.1 Detecting matches . . . 4
- 7.3.2 Extracting matches . . . 5
- 7.3.3 Grouped matches . . . 5
- 7.3.4 Replacing and splitting . . . 6

7.1 String basics

Motivation. Much real data arrives as text: names, addresses, free-form survey answers, codes that pack several pieces of information into one field. Before such data can be analysed, the information has to be found and pulled out of the strings, and stringr with regular expressions is the tool for that.

`stringr` provides consistent functions for strings, all starting with `str_`. It is part of the core tidyverse but can be loaded alone with `library(stringr)`.

Strings are written with double or single quotes; use single quotes when the string contains double quotes:

```
string1 <- "This is a string"
string2 <- 'If I want to include a "quote" inside a string, I use single quotes'
```

If you forget the closing quote, the console shows the continuation prompt `+` and waits; press Esc and try again.

Escapes. The backslash `\` escapes a special character: `"\""` is a double quote and `"\\"` a literal backslash. The printed representation of a string shows the escapes; `writeLines()` shows the raw contents:

```
x <- c("\"", "\\")
x
```

```
## [1] "\"" "\\"
```

```
writeLines(x)
```

```
## "
## \
```

Other special characters are `"\n"` (newline) and `"\t"` (tab); `?"'"` lists them all. Any Unicode character can be written as `"\u"` followed by its code, e.g. `"\uc804\uc0b0\ud1b5\uacc4"` is the course name in Hangul. Several strings are stored in a character vector, `c("one", "two", "three")`.

7.1.1 Length, combining, subsetting

```
str_length(c("a", "R for data science", NA))
```

```
## [1]  1 18 NA
```

```
str_c("x", "y", sep = ", ")
```

```
## [1] "x, y"
```

```
str_c("prefix-", c("a", "b", "c"), "-suffix")     # vectorised, recycles
```

```
## [1] "prefix-a-suffix" "prefix-b-suffix" "prefix-c-suffix"
```

```
str_c(c("x", "y", "z"), collapse = ", ")          # collapse a vector into one string
```

```
## [1] "x, y, z"
```

`str_c()` drops objects of length 0, which is convenient with `if`: in `str_c("Good ", time_of_day, " ", name, if (birthday) " and HAPPY BIRTHDAY", ".")` the clause disappears when `birthday` is `FALSE`. Missing values are contagious; `str_replace_na()` prints them as `"NA"`.

`str_sub(x, start, end)` extracts characters by position (inclusive); negative positions count from the end, and positions out of range are not an error:

```
x <- c("Apple", "Banana", "Pear")
str_sub(x, 1, 3)
```

```
## [1] "App" "Ban" "Pea"
```

```
str_sub(x, -3, -1)
```

```
## [1] "ple" "ana" "ear"
```

```
str_sub("a", 1, 5)
```

```
## [1] "a"
```

```
str_sub(x, 1, 1) <- str_to_lower(str_sub(x, 1, 1))   # assignment form
x
```

```
## [1] "apple"  "banana" "pear"
```

Positions count characters, not bytes, so `str_sub()` works for Hangul as well.

7.2 Regular expressions

Motivation. Fixed strings are too rigid for real text: we want "any digit", "a word at the start", "a repeated letter". Regular expressions are a small language for describing such patterns; they are terse, but a handful of rules covers most needs.

`str_view()` shows the matches of a pattern, marking each with `< >` (older code uses `str_view_all()`, now deprecated).

7.2.1 Basic matches

The simplest pattern matches an exact string; `.` matches any character except a newline:

```
x <- c("apple", "banana", "pear")
str_view(x, "an")
```

```
## [2] | b<an><an>a
```

```
str_view(x, ".a.")
```

```
## [2] | <ban>ana
## [3] | p<ear>
```

To match a literal `.` the regular expression needs the escape `\.`. But regular expressions are written as strings, and in a string the backslash itself must be escaped, so the R string is `"\\."`. Likewise a literal backslash needs the regular expression `\\`, i.e. the string `"\\\\"`:

```
str_view(c("abc", "a.c", "bef"), "a\\.c")
```

```
## [2] | <a.c>
```

```
writeLines("a\\b")
```

```
## a\b
```

```
str_view("a\\b", "\\\\")
```

```
## [1] | a<\>b
```

7.2.2 Anchors and character classes

`^` matches the start of the string and `$` the end ("if you begin with power (`^`), you end up with money (`$`)"). Both together force a complete match:

```
x <- c("apple pie", "apple", "apple cake", "pine apple")
str_view(x, "^apple$")
```

```
## [2] | <apple>
```

Special patterns: `\d` any digit, `\s` any whitespace, `[abc]` one of a, b, c, `[^abc]` anything except a, b, c. In a string these are written `"\\d"` and so on. Alternation `|` picks one of several alternatives, with parentheses to delimit them:

```
str_view(c("grey", "gray"), "gr(e|a)y")
```

```
## [1] | <grey>
## [2] | <gray>
```

7.2.3 Repetition

`?` means 0 or 1, `+` 1 or more, `*` 0 or more; `{n}` exactly n, `{n,}` n or more, `{,m}` at most m, `{n,m}` between n and m. Repetition is **greedy** by default (it matches the longest possible string); a `?` after the repetition makes it **lazy** (shortest match).

```
x <- "1888 is the longest year in Roman numerals: MDCCCLXXXVIII"
str_view(x, "CC+")
```

```
## [1] | 1888 is the longest year in Roman numerals: MD<CCC>LXXXVIII
```

```
str_view(x, "C[LX]+")
```

```
## [1] | 1888 is the longest year in Roman numerals: MDCC<CLXXX>VIII
```

```
str_view(x, "C{2,3}")
```

```
## [1] | 1888 is the longest year in Roman numerals: MD<CCC>LXXXVIII
```

```
str_view(x, "C{2,3}?")
```

```
## [1] | 1888 is the longest year in Roman numerals: MD<CC>CLXXXVIII
```

7.2.4 Grouping and backreferences

Parentheses also define **groups**, which can be referred to later in the pattern as `\1`, `\2`, ... Fruits containing a repeated pair of letters:

```
str_subset(fruit, "(..)\\1")
```

```
## [1] "banana"      "coconut"     "cucumber"    "jujube"      "papaya"      "salal berry"
```

7.3 Tools

Motivation. With patterns in hand we can do the real tasks: test which strings match, pull out the matching pieces, replace them, or split strings apart. Each task is one stringr function.

7.3.1 Detecting matches

`str_detect()` returns a logical vector, so counts and proportions follow from `sum()` and `mean()`. `words` is a vector of 980 common English words.

```
str_detect(c("apple", "banana", "pear"), "e")
```

```
## [1]  TRUE FALSE  TRUE
```

```
sum(str_detect(words, "^t"))          # how many words start with t?
```

```
## [1] 65
```

```
mean(str_detect(words, "[aeiou]$"))   # proportion ending with a vowel
```

```
## [1] 0.2765306
```

```
str_subset(words, "x$")                # same as words[str_detect(words, "x$")]
```

```
## [1] "box" "sex" "six" "tax"
```

Inside a data frame, use `filter(str_detect(word, "x$"))`. `str_count()` counts matches per string, e.g. the number of vowels:

```
tibble(word = words) %>%
  mutate(vowels = str_count(word, "[aeiou]"),
         consonants = str_count(word, "[^aeiou]"))
```

```
## # A tibble: 980 x 3
##   word     vowels consonants
##   <chr>     <int>      <int>
## 1 a             1          0
## 2 able          2          2
## 3 about         3          2
## 4 absolute      4          4
## 5 accept        2          4
## # i 975 more rows
```

Complex conditions are often easier with several `str_detect()` calls combined by logical operators than with a single regular expression: words without vowels are `!str_detect(words, "[aeiou]")`.

7.3.2 Extracting matches

`str_extract()` returns the first match of each string (`NA` if none), `str_extract_all()` all matches as a list (or a matrix with `simplify = TRUE`). The `sentences` data contain 720 short sentences; to find those mentioning a colour:

```
colours <- c("red", "orange", "yellow", "green", "blue", "purple")
colour_match <- str_c(colours, collapse = "|")
has_colour <- str_subset(sentences, colour_match)
head(str_extract(has_colour, colour_match))
```

```
## [1] "blue" "blue" "red"  "red"  "red"  "blue"
```

(This simple pattern also matches "red" inside "flickered"; adding word boundaries `\b` around the alternatives fixes that [Supplement].)

7.3.3 Grouped matches

`str_match()` returns the full match and each group as columns of a matrix. A rough pattern for a noun phrase, "a" or "the" followed by a word:

```
noun <- "(a|the) ([^ ]+)"
sentences %>% str_subset(noun) %>% head(4) %>% str_match(noun)
```

```
##      [,1]         [,2]  [,3]
## [1,] "the smooth" "the" "smooth"
## [2,] "the sheet"  "the" "sheet"
## [3,] "the depth"  "the" "depth"
## [4,] "a chicken"  "a"   "chicken"
```

For a tibble, `tidyr::extract()` does the same and names the groups as new columns:

```
tibble(sentence = sentences) %>%
  extract(sentence, c("article", "noun"), "(a|the) ([^ ]+)", remove = FALSE)
```

```
## # A tibble: 720 x 3
##   sentence                                 article noun
##   <chr>                                    <chr>   <chr>
```

```
## 1 The birch canoe slid on the smooth planks.  the     smooth
## 2 Glue the sheet to the dark blue background. the     sheet
## 3 It's easy to tell the depth of a well.      the     depth
## 4 These days a chicken leg is a rare dish.    a       chicken
## 5 Rice is often served in round bowls.        <NA>    <NA>
## # i 715 more rows
```

7.3.4 Replacing and splitting

`str_replace()` replaces the first match and `str_replace_all()` all matches; a named vector performs several replacements, and backreferences reorder groups:

```
x <- c("1 house", "2 cars", "3 people")
str_replace_all(x, c("1" = "one", "2" = "two", "3" = "three"))
```

```
## [1] "one house"    "two cars"     "three people"
```

```
sentences %>%
  str_replace("([^ ]+) ([^ ]+) ([^ ]+)", "\\1 \\3 \\2") %>%   # swap 2nd and 3rd words
  head(3)
```

```
## [1] "The canoe birch slid on the smooth planks."
## [2] "Glue sheet the to the dark blue background."
## [3] "It's to easy tell the depth of a well."
```

`str_split()` splits strings into pieces (a list, since each string can give a different number of pieces); `boundary("word")` splits on word boundaries rather than a single character:

```
"a|b|c|d" %>% str_split("\\|")
```

```
## [[1]]
## [1] "a" "b" "c" "d"
```

```
x <- "This is a sentence.  This is another sentence."
str_split(x, boundary("word"))[[1]]
```

```
## [1] "This"     "is"       "a"        "sentence" "This"     "is"       "another"
## [8] "sentence"
```

`str_locate()` gives the start and end positions of matches, which can be passed to `str_sub()`.